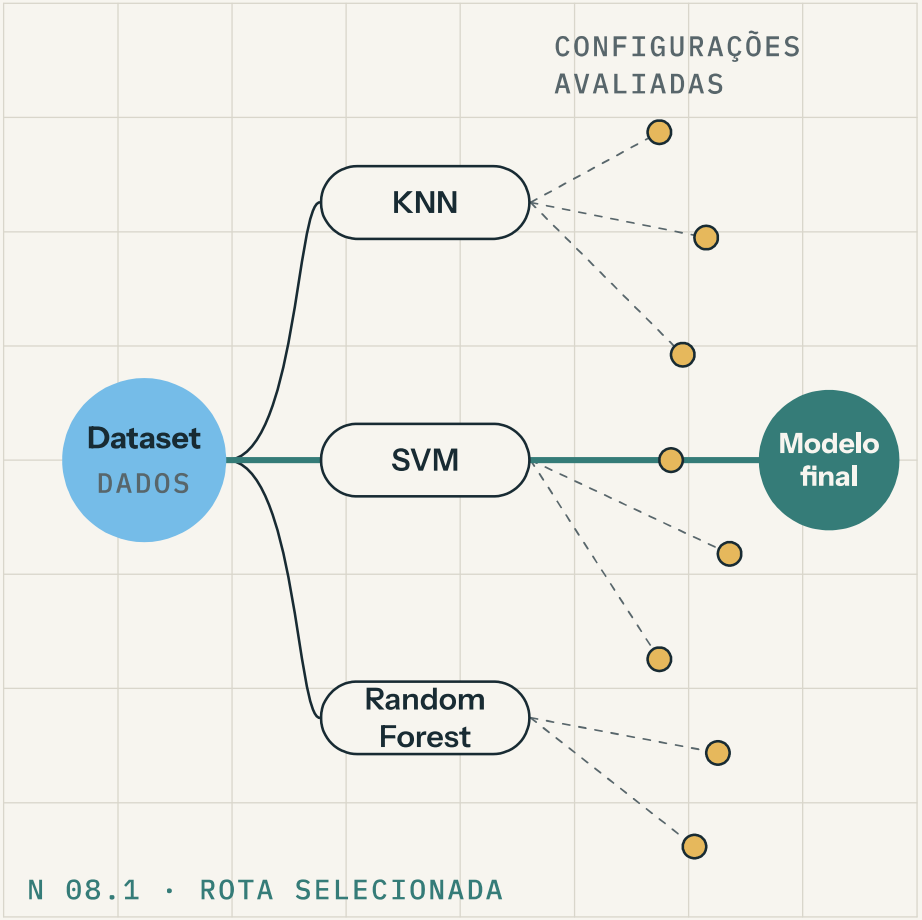
◇ AUTOML ATLAS · MAPPING THE SEARCH SPACE

MÓDULO 8 · AUTOML, FERRAMENTA POR FERRAMENTA · AULA 8.1

O que o AutoML resolve o problema CASH

Como selecionar algoritmos, otimizar hiperparâmetros e combinar modelos dentro de um orçamento computacional.

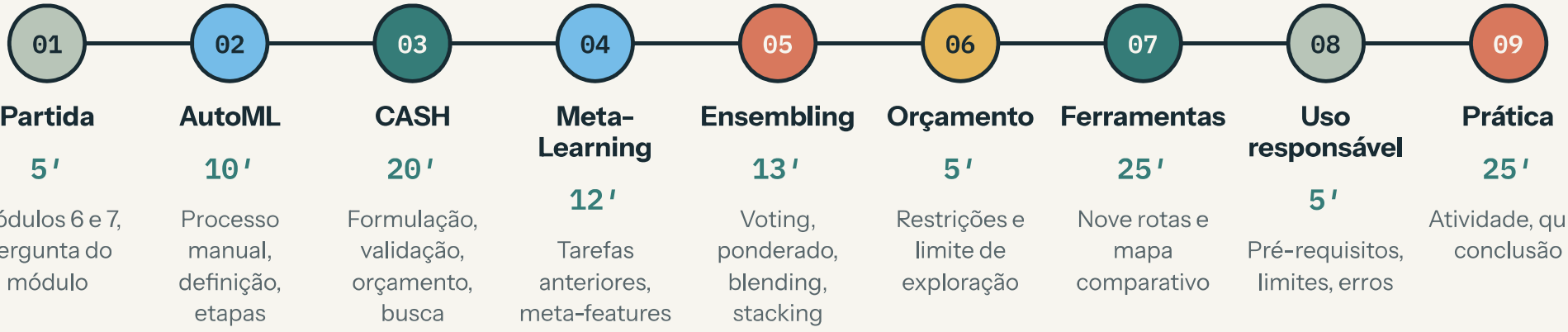
Seleção de algoritmos | Hiperparâmetros | Meta-Learning | Ensembling



◇ ROTA 01 · ITINERÁRIO

PARTIDA

Rota da aula: nove paradas em 120 minutos



◇ Hoje: o que cada ferramenta automatiza e por quê. As APIs ficam para as aulas 8.2 a 8.10.

Objetivos de aprendizagem

PROBLEMA

- O que é AutoML e o que ele não é
- Etapas do ML que podem ser automatizadas
- O problema CASH e sua formulação

MECANISMOS

- Seleção de algoritmos e otimização de hiperparâmetros
- Busca de pipelines
- Meta-Learning e meta-features
- Ensembling automático
- Orçamento computacional

ECOSSISTEMA

- Diferenças entre as nove ferramentas do módulo
- Leitura crítica de versões e documentação

JULGAMENTO

- Limitações e riscos
- Quando usar e quando evitar AutoML
- Por que os Módulos 6 e 7 são pré-requisitos

Pergunta-guia: que parte do problema esta ferramenta automatiza, com que estratégia e com que custo?

◇ ROTA 01 · CONEXÃO

PARTIDA

Do Módulo 6 ao Módulo 8

MÓDULO 6 **Como os algoritmos funcionam**

Regressão · Classificação · KNN · Árvores de Decisão · K-Means · PCA · Perceptron · MLP · Backpropagation

→

MÓDULO 7 **Muitos algoritmos, muitas configurações**

Regularização · SVM · Random Forest · XGBoost · LightGBM · CatBoost · Grid Search · Random Search · Optuna

---

MÓDULO 8 **Como escolher automaticamente uma boa combinação entre algoritmos e hiperparâmetros?**

Exemplo: previsão de compra em uma loja online

A loja quer prever se o cliente comprará nos próximos 30 dias.

```
idade          regiao
visitas        tempo_site
itens_carrinho
compras_anteriores

alvo → comprou (0/1)
```

Esse caso acompanha toda a aula.

UMA VEZ

1. Analisar o dataset
2. Tratar ausentes
3. Preparar características

LAÇO EXPERIMENTAL

↺ DEZENAS DE VOLTAS

4. Escolher algoritmo
5. Definir hiperparâmetros
6. Treinar
7. Validar
8. Comparar métricas
9. Modificar configurações
10. Repetir experimentos

SAÍDA

11. Selecionar o modelo

Custos do processo manual

01 **Tempo**
Cada volta do laço leva minutos ou horas de trabalho humano e de máquina.

02 **Computação**
Cada configuração exige k treinos na validação cruzada.

03 **Memória**
Modelos grandes e datasets inteiros disputam a RAM disponível.

04 **Experimentação**
Sem registro sistemático, perde-se o histórico do que já foi testado.

05 **Conhecimento técnico**
Escolher faixas razoáveis de hiperparâmetros exige experiência.

06 **Reprodutibilidade**
Buscas feitas em células fora de ordem são difíceis de repetir.

◇ O AutoML reduz o tempo humano e organiza os experimentos, mas pode aumentar o custo computacional.

◇ ROTA 02 · PROCESSO MANUAL

AUTOML

Quantos candidatos existem?

ALGORITMO	GRADE DE HIPERPARÂMETROS	TOTAL
Reg. Logística	`C(10) × penalty(2)`	**20**
KNN	`k(10) × weights(2) × metric(3)`	**60**
SVM linear	`C(10)`	**10**
SVM rbf / poly	`C(10) × gamma(10) × kernel(2)`	**200**
Random Forest	`n_est(5) × depth(6) × feat(3)`	**90**
Gradient Boosting	`n_est(5) × lr(5) × depth(6)`	**150**
Soma	as grades se somam entre algoritmos	**530**



530 configurações × 5 folds.
gamma só existe para kernels rbf e poly.

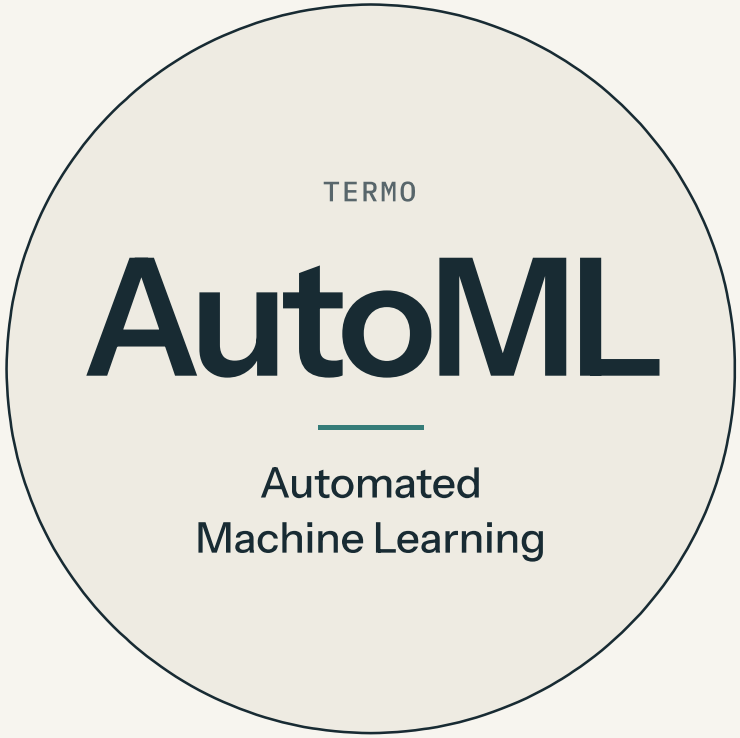
DADOS ILUSTRATIVOS Grades pequenas escolhidas para o exemplo.

O que é AutoML

Automated Machine Learning: técnicas e ferramentas que automatizam **partes** do desenvolvimento de modelos de Machine Learning.

AUTOML NÃO É

- uma única biblioteca
- apenas Grid Search
- sinônimo de CASH: o CASH é um problema dentro do AutoML



◇ ROTA 02 · ESCOPO

AUTOML

Etapas que o AutoML pode automatizar

01

Pré-processamento

Deixa os dados utilizáveis: imputação, codificação, escalonamento.

02

Engenharia de características

Cria ou seleciona variáveis: razões, agregações, interações.

03

Seleção de algoritmos

Escolhe a família de modelo entre candidatos.

04

Otimização de hiperparâmetros

Ajusta a configuração de um algoritmo.

05

Busca de pipelines

Escolhe componentes e a ordem em que se encadeiam.

06

Seleção de modelos

Escolhe entre modelos já treinados pelo critério de validação.

07

Ensembling

Combina vários modelos em uma previsão.

08

Busca de arquitetura neural

Escolhe a estrutura de uma rede: camadas, larguras, conexões.

◇ O que cada ferramenta automatiza varia. Nenhuma cobre todas as etapas com a mesma profundidade.

Rota manual e rota com AutoML

MANUAL



COM AUTOML



◇ As etapas continuam existindo. Muda quem percorre o laço e como ele é registrado.

Decisões que continuam com o usuário

◇ Problema

EXEMPLO DA LOJA

Classificação binária: comprará em 30 dias?

◇ Métrica

EXEMPLO DA LOJA

ROC AUC, ou precisão entre os 500 clientes mais prováveis

◇ Validação

EXEMPLO DA LOJA

Estratificada; temporal se houver data da visita

◇ Orçamento

EXEMPLO DA LOJA

30 minutos, 8 GB de RAM, sem GPU

◇ Restrições

EXEMPLO DA LOJA

Prever em menos de 50 ms por cliente

◇ Critério de sucesso

EXEMPLO DA LOJA

Superar a linha de base com margem definida antes

◇ O AutoML otimiza fielmente o que recebe. Se a métrica ou a validação estiverem erradas, ele otimiza o erro.

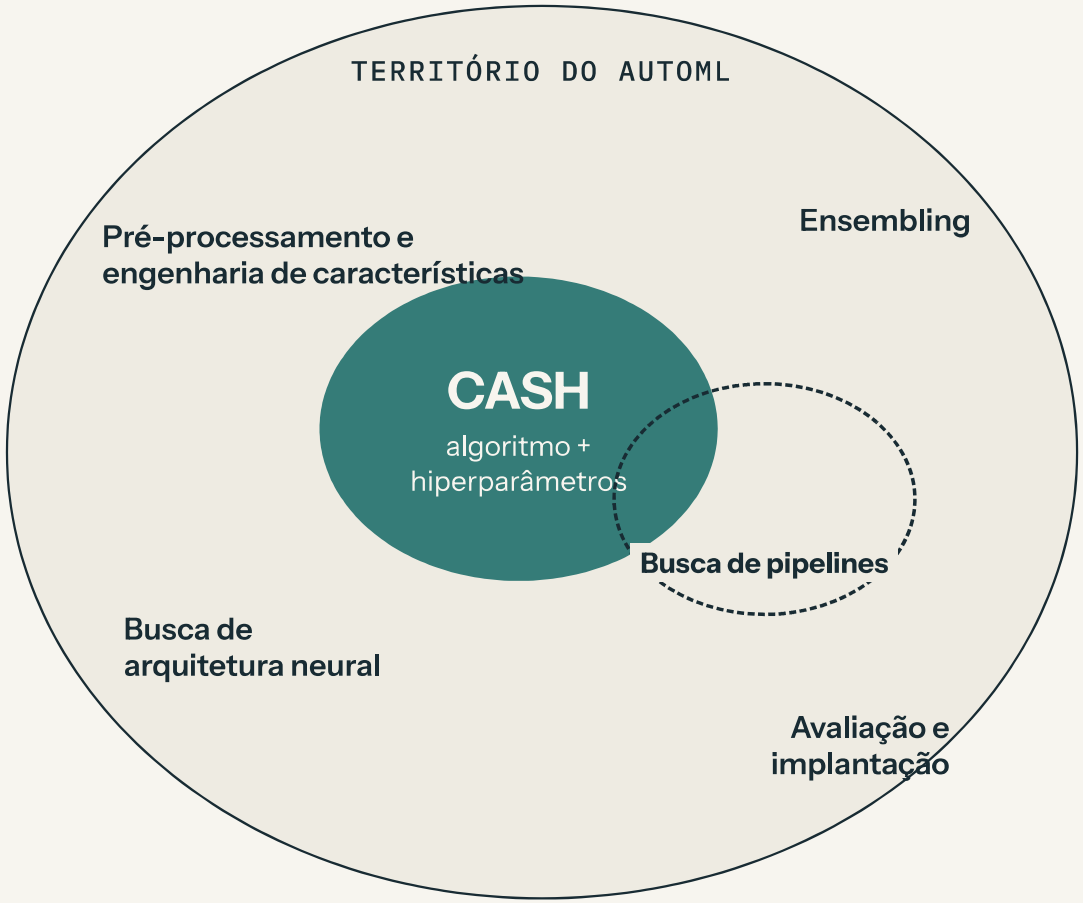
CASH: um problema dentro do AutoML

C — **Combined**
Combinada

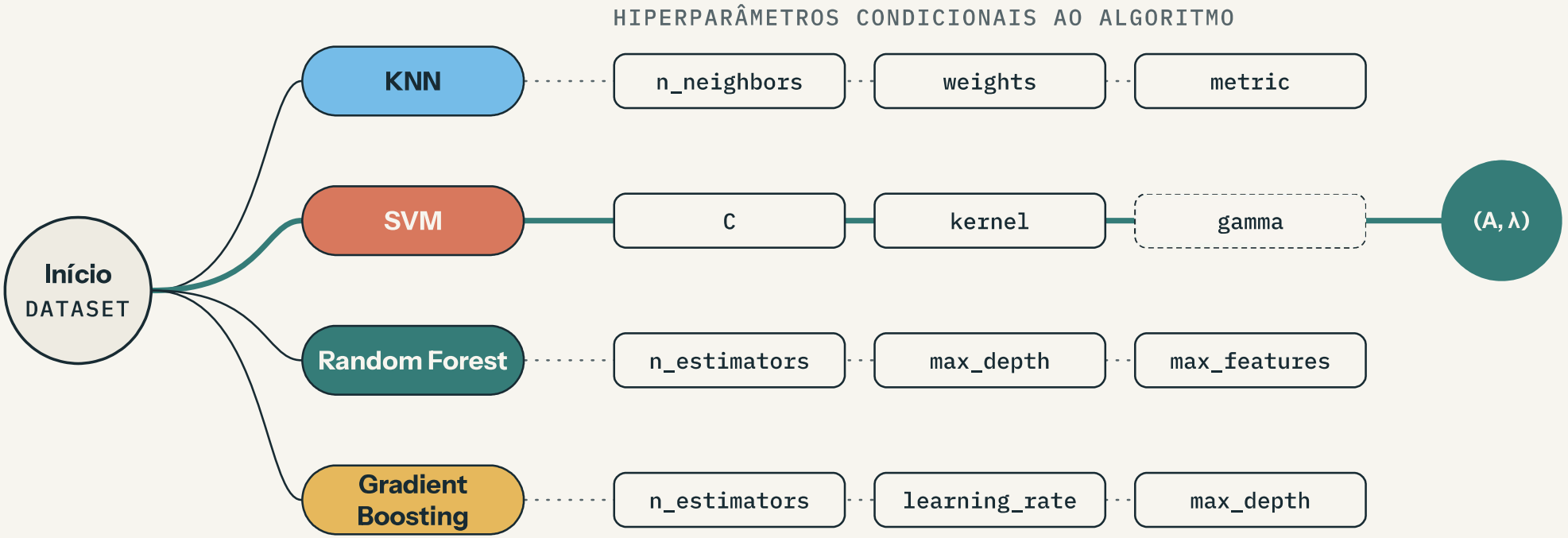
A — **Algorithm**
de Algoritmos

S — **Selection**
Seleção

H — **Hyperparameter optimization**
e Otimização de Hiperparâmetros



CASH como mapa de rotas: cada algoritmo tem seu próprio espaço



ROTA DESTACADA `(SVM, C=1, kernel=rbf, gamma=0.01)`

* gamma só existe para kernels rbf, poly e sigmoid.

CASH como problema de minimização

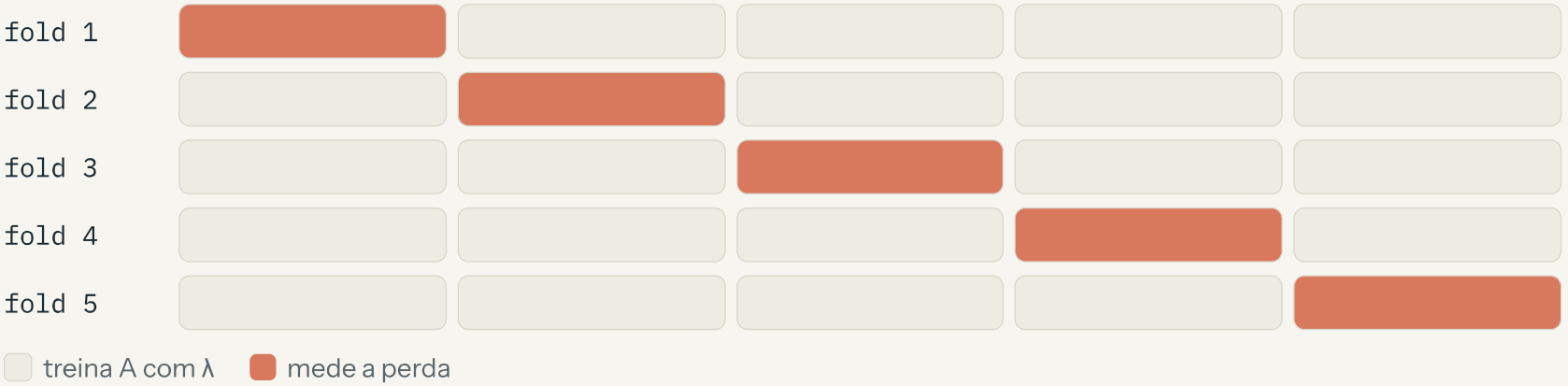
$$(A^*, \lambda^*) = \underset{A \in \mathcal{A},\, \lambda \in \Lambda_A}{\arg\min}\ \mathcal{L}_{\text{val}}(A, \lambda)$$

A algoritmo candidato	$\mathcal{A}$ conjunto de algoritmos disponíveis	λ vetor de hiperparâmetros	Λ_A espaço de hiperparâmetros do algoritmo A
$\mathcal{L}_{\text{val}}$ perda estimada na validação	A^* algoritmo selecionado	λ^* configuração selecionada	$\arg\min$ o par que minimiza, não o valor mínimo

Representação didática do problema, não a descrição interna de uma ferramenta específica.

Como a perda de validação é estimada

$$\mathcal{L}_{\text{val}}(A, \lambda) = \frac{1}{k} \sum_{i=1}^{k} \mathcal{L}(A_{\lambda}, \mathcal{D}^{(i)}_{\text{treino}}, \mathcal{D}^{(i)}_{\text{val}})$$



Teste

FORA DA BUSCA

◇ ROTA 03 · FORMULAÇÃO

CASH

Minimizar uma perda ou maximizar uma métrica

$$(A^*, \lambda^*) = \underset{A \in \mathcal{A},\, \lambda \in \Lambda_A}{\arg\max}\ Score_{\text{val}}(A, \lambda)$$

equivale a minimizar $\mathcal{L} = -Score$ ou $\mathcal{L} = 1 - Score$

MAXIMIZAR

ROC AUC F1

MINIMIZAR

Log Loss Erro quadrático

CONVENÇÃO DO SCIKIT-LEARN

`scoring="neg_log_loss"`

Perdas aparecem com sinal negativo para que maior seja sempre melhor.

◇ ROTA 03 · ORÇAMENTO NA FORMULAÇÃO

CASH

O orçamento limita a busca inteira

RESTRIÇÃO

$$\sum_{t=1}^{T} Custo(A_t, \lambda_t) \leq B$$

RESULTADO

$$(\hat{A}, \hat{\lambda}) = \underset{(A,\lambda) \in \mathcal{H}_B}{\arg\min}\ \mathcal{L}_{\text{val}}(A, \lambda)$$

$\mathcal{H}_B$ é o conjunto de pares que couberam no orçamento. O par devolvido é o melhor encontrado, não necessariamente o ótimo global.

B PODE SER MEDIDO EM

- Tempo de relógio
- Horas de CPU ou GPU
- Memória disponível
- Número de avaliações

Versão anterior corrigida: o limite vale para a soma das avaliações, não para cada uma.

Comparando algoritmos e configurações

DADOS ILUSTRATIVOS

ALGORITMO	CONFIGURAÇÃO	ROC AUC VAL.
KNN	k=5	0,82
KNN	k=9	0,85
SVM	C=1, RBF	0,88
SVM	C=10, RBF	0,86
Random Forest	100 árvores	0,87
Random Forest	**300 árvores**	**0,89**

1. Compara configurações dentro de cada algoritmo
2. Compara todos os pares entre algoritmos
3. Devolve (Random Forest, 300 árvores)

0,88 e 0,89 podem estar dentro do ruído da validação. O desempenho final exige o teste independente.

Representando o espaço de busca em Python

```
from sklearn.model_selection import StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

espaco = [
    (KNeighborsClassifier(),
     {"m__n_neighbors": [5, 9]}),
    (SVC(),
     {"m__C": [1, 10], "m__kernel": ["rbf"]}),
    (RandomForestClassifier(random_state=0),
     {"m__n_estimators": [100, 300]}),
]

cv = StratifiedKFold(n_splits=5, shuffle=True,
                     random_state=0)
```

- Cada par tem sua própria grade: espaço condicional.
- m__ endereça a etapa chamada m do Pipeline.
- Escalonamento dentro do Pipeline evita vazamento entre folds.
- Folds estratificados e semente fixa.

Uma busca ingênua com orçamento

```
import time
from sklearn.base import clone
from sklearn.model_selection import ParameterGrid, cross_val_score
from sklearn.pipeline import Pipeline

def cash_ingenuo(espaco, X, y, cv, orcamento_s=600):
    inicio, melhor = time.time(), (None, float("-inf"))
    for modelo, grade in espaco:
        for cfg in ParameterGrid(grade):
            if time.time() - inicio > orcamento_s:
                return melhor          # orçamento esgotado
            pipe = Pipeline([("esc", StandardScaler()),
                             ("m", clone(modelo))]).set_params(**cfg)
            auc = cross_val_score(pipe, X, y, cv=cv,
                                  scoring="roc_auc").mean()
            if auc > melhor[1]:
                melhor = (pipe, auc)
    return melhor
```

O QUE FALTA AQUI

- Aprender com avaliações anteriores
- Interromper cedo treinos ruins
- Usar experiência de outros datasets
- Combinar os melhores modelos

Corrigido: return encerra a busca inteira e clone evita reaproveitar o mesmo objeto.

Seleção de algoritmos: famílias com vieses diferentes

Lineares

fronteiras aproximadamente lineares; rápidos e interpretáveis

Árvores

partições do espaço por regras

Random Forest

bagging de árvores reduz variância

Boosting

árvores em sequência reduzem viés

SVM

margem máxima; kernels para fronteiras não lineares

Redes neurais

aprendem representações; fortes em imagem e texto

Comparar com padrões

Cada família com configuração padrão

EX.: LAZYPREDICT

Família como hiperparâmetro

A escolha entra no espaço de busca

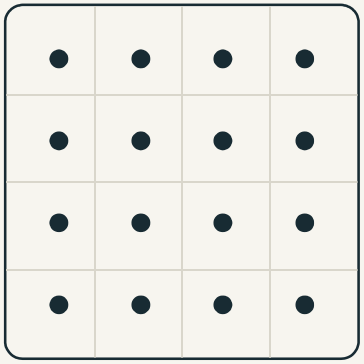
EX.: AUTO-SKLEARN

Portfólio e prioridade

Começar pelo que costuma funcionar

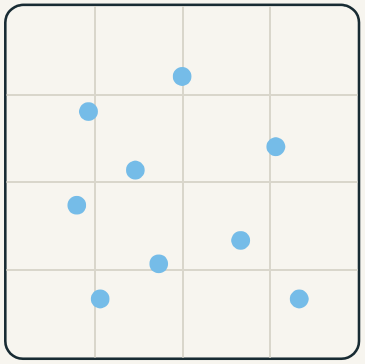
EX.: AUTOGLUON, FLAML

Estratégias de otimização de hiperparâmetros



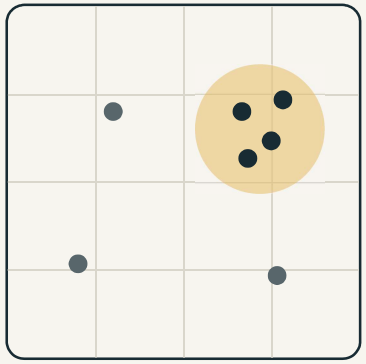
Grid Search

Todos os pontos de uma grade fixa



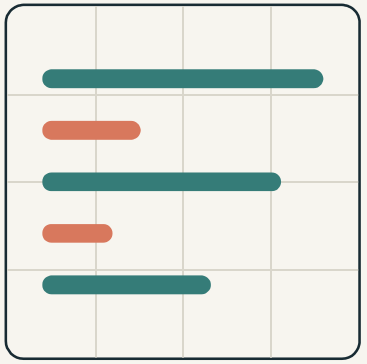
Random Search

Pontos sorteados; explora mais valores de cada eixo



Adaptativa · TPE

Usa resultados anteriores para escolher o próximo ponto



Pruning

Interrompe cedo avaliações pouco promissoras

◇ Todas competem pelo mesmo orçamento. Cada ferramenta de AutoML incorpora estratégias diferentes.

◇ ROTA 03 · PIPELINES

CASH

Busca de pipelines: escolher componentes, não só valores

Imputação	Codificação	Escalonamento	Seleção	Modelo
mediana	**one-hot**	padrão	variância	KNN
média	ordinal	min-max	PCA	SVM
mais frequente		**nenhum**	**nenhuma**	**Boosting**

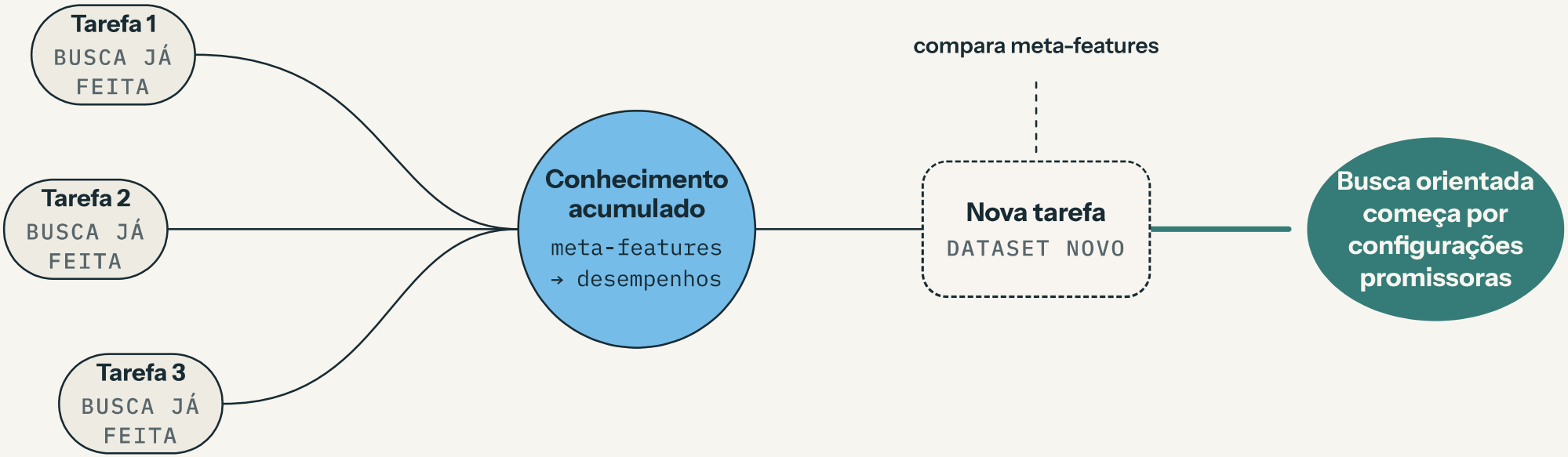
ROTA ESCOLHIDA

`mediana → one-hot → nenhum → nenhuma → Boosting`

ESTRUTURA

Auto-Sklearn: estrutura fixa. TPOT: estruturas livres, em grafo.

Meta-Learning: aprender com tarefas anteriores



Recomendação de algoritmos

Configurações iniciais (warm start)

Portfólio aprendido offline

Meta-features: descrever um dataset com números

SIMPLES

- número de amostras
- número de atributos
- número de classes
- proporção de categóricas

DISTRIBUIÇÃO

- proporção da classe minoritária
- proporção de valores ausentes

ESTATÍSTICAS

- assimetria e curtose médias
- correlação média entre atributos

LANDMARKERS

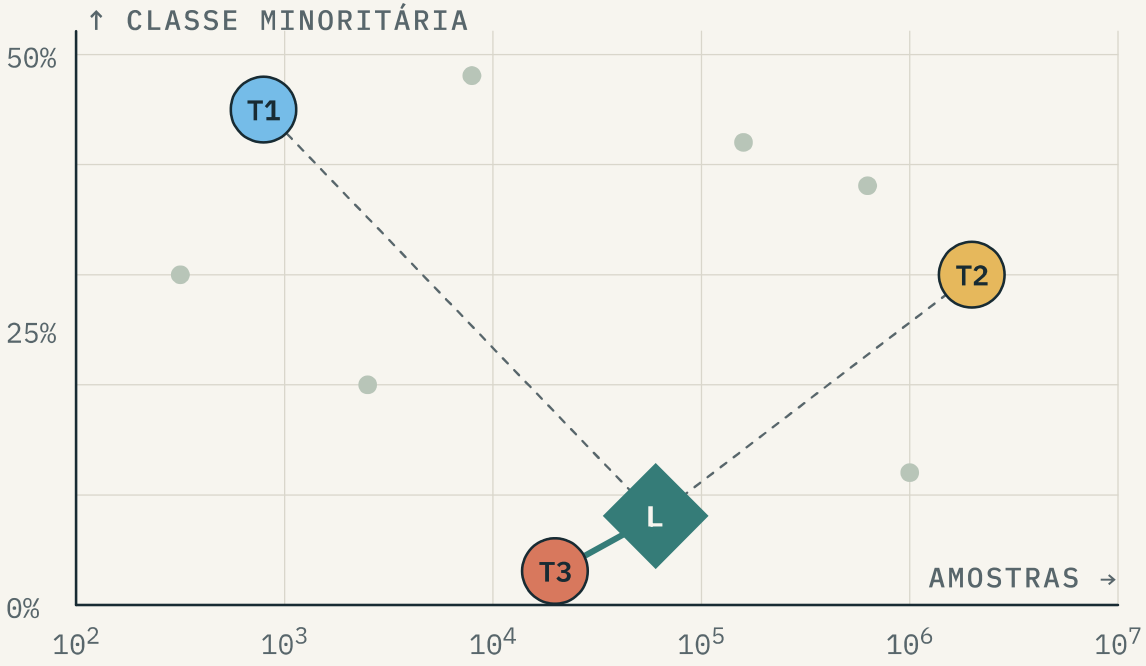
- desempenho de modelos simples, como uma árvore de profundidade 1

LOJA ONLINE — **DADOS ILUSTRATIVOS**

amostras	60.000
atributos	6
classes	2
minoritária	8%
ausentes	4%
categóricas	1 de 6
árvore prof. 1	AUC 0,71

Normalizado, o vetor permite medir a distância até datasets anteriores: um KNN entre datasets.

Meta-Learning na prática: o vizinho mais próximo orienta o início



T1 **Pequeno, poucas características**
n=800 · d=12 · 45%
Bom: SVM RBF, Reg. Logística

T2 **Grande, muitas características**
n=2M · d=900 · 30%
Bom: Boosting com amostragem de colunas

T3 **Tabular desbalanceado**
n=20k · d=40 · 3%
Bom: Boosting com pesos de classe, RF balanceada

Meta-Learning não prevê o melhor modelo. Ele usa experiências anteriores para orientar a busca.

◇ ROTA 04 · COMPARAÇÃO

META-LEARNING

Meta-Learning e otimização bayesiana não são a mesma coisa

	OTIMIZAÇÃO BAYESIANA	META-LEARNING
Pergunta	Onde avaliar o próximo ponto?	Por onde começar?
Fonte de informação	Avaliações já feitas nesta tarefa	Buscas feitas em outras tarefas
Quando atua	Durante a busca	Antes ou no início da busca
Modelo usado	Substituto da perda (GP, floresta, TPE)	Semelhança entre meta-features ou portfólio
Sem histórico	Funciona normalmente	Não tem o que transferir

COMBINAÇÃO **Meta-Learning escolhe o início** → **Otimização bayesiana continua** ex.: Auto-Sklearn, Auto-PyTorch

Por que combinar modelos: erros diferentes se compensam

DEZ EXEMPLOS DE VALIDAÇÃO

DADOS ILUSTRATIVOS

	1	2	3	4	5	6	7	8	9	10	ACERTOS
Modelo A	✓	✗	✓	✓	✓	✓	✗	✓	✓	✓	8/10
Modelo B	✓	✓	✓	✗	✓	✓	✗	✓	✓	✓	8/10
Modelo C	✓	✓	✓	✓	✗	✓	✓	✓	✗	✓	8/10
Maioria	✓	✓	✓	✓	✓	✓	✗	✓	✓	✓	9/10

O ganho vem da diversidade: o ensemble só erra onde a maioria erra ao mesmo tempo.

O custo: modelo final maior, inferência mais lenta e explicação mais difícil.

Hard voting e soft voting

HARD VOTING · VOTO POR CLASSE

$$\hat{y} = \arg\max_{c} \sum_{m=1}^{M} \mathbb{1}[\hat{y}_m = c]$$

Conta votos. Ignora a confiança de cada modelo.

SOFT VOTING · MÉDIA DE PROBABILIDADES

$$\hat{y} = \arg\max_{c} \frac{1}{M} \sum_{m=1}^{M} p_m(c \mid x)$$

Usa a confiança. Exige probabilidades bem calibradas.

DADOS ILUSTRATIVOS

P(compra) = `0,45 · 0,45 · 0,95`

HARD

votos 0, 0, 1 → **não compra**

SOFT

média 0,62 → **compra**

Weighted ensemble: pesos aprendidos na validação

$$\hat{p}(c \mid x) = \sum_{m=1}^{M} w_m \, p_m(c \mid x)$$

$$w_m \geq 0, \sum w_m = 1$$

Soft voting é o caso particular $w_m = 1/M$.

Risco: com muitos modelos e pouca validação, os pesos sobreajustam a própria validação.

SELEÇÃO GULOSA DE CARUANA — DADOS ILUSTRATIVOS

passo 1	adiciona	RF
passo 2	adiciona	GB
passo 3	adiciona	RF
passo 4	adiciona	SVM

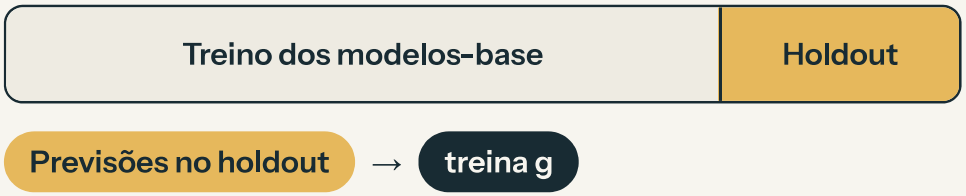
RF 0,50 GB 0,25 SVM 0,25

Com reposição: a frequência de cada modelo vira seu peso.

Blending e stacking: um meta-modelo sobre previsões

$$\hat{y} = g(\, f_1(x), f_2(x), \ldots, f_M(x)\,)$$

◇ Blending



Previsões no holdout → treina g

Barato. Modelos-base e combinador veem menos dados.

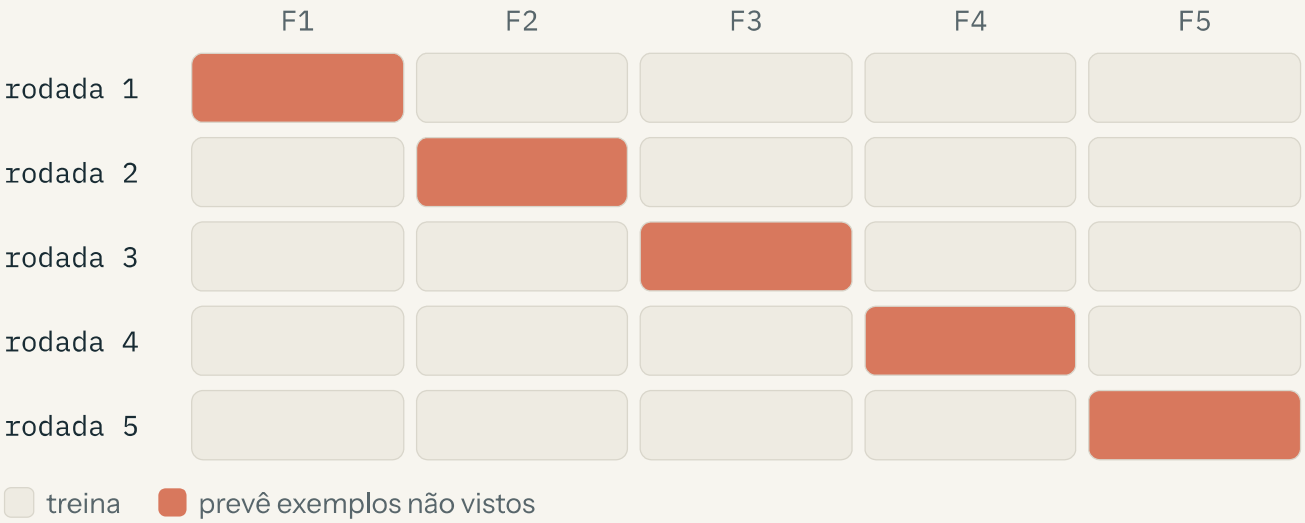
◇ Stacking

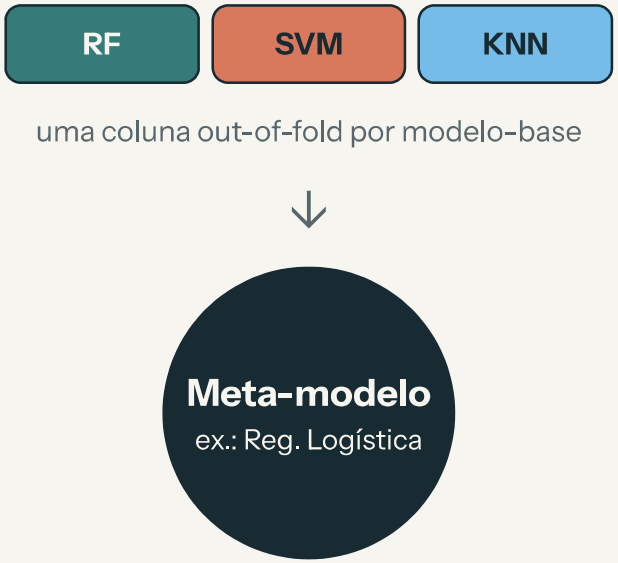


Previsões out-of-fold → treina g

Usa todos os exemplos sem vazamento. Custa k treinos por modelo-base.

Previsões out-of-fold evitam vazamento no stacking





Custo e risco de cada método

MÉTODO	TREINO EXTRA	DADOS PARA COMBINAR	INFERÊNCIA	RISCO PRINCIPAL
Hard voting	nenhum	—	`M modelos`	modelos fracos pesam igual
Soft voting	nenhum	—	`M modelos`	probabilidades mal calibradas
Weighted	avaliar combinações	validação ou OOF	`M modelos`	sobreajustar a validação
Blending	treinar g	holdout	`M + g`	menos dados para cada etapa
Stacking	k × M treinos + g	previsões OOF	`M + g (ou k × M)`	custo; vazamento sem OOF

◇ Nem todas as ferramentas executam todos esses métodos. LazyPredict não faz nenhum; FLAML faz stacking só quando solicitado.

Restrições que moldam a busca e o resultado

LIMITAM A BUSCA

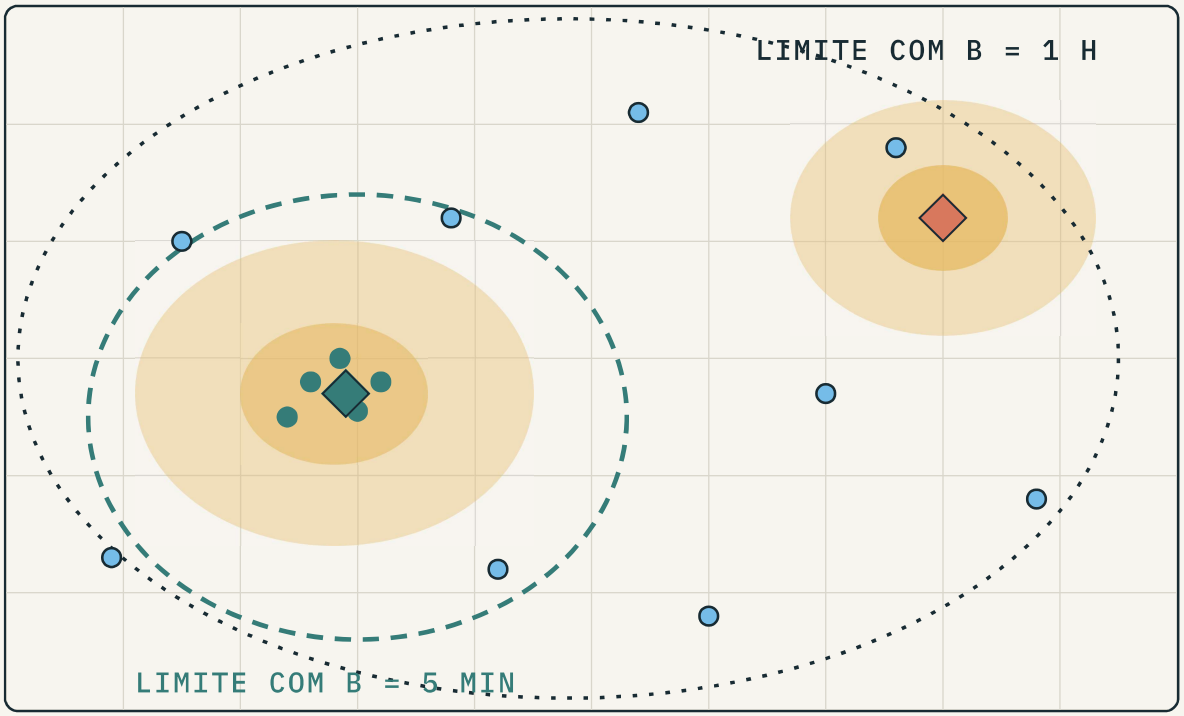
- **Tempo**
  limite de relógio para a busca
- **CPU**
  núcleos em paralelo
- **GPU**
  necessária para redes neurais de porte
- **Memória**
  tamanho de dados e modelos na RAM

LIMITAM O RESULTADO E O CUSTO

- **Número de experimentos**
  teto de avaliações
- **Custo de treinamento**
  preço em nuvem por hora
- **Latência de inferência**
  tempo por previsão em produção
- **Tamanho do modelo**
  disco, memória, dispositivos móveis

◇ A solução encontrada depende das restrições. Restrições de produção devem ser declaradas antes da busca.

O orçamento como limite de exploração

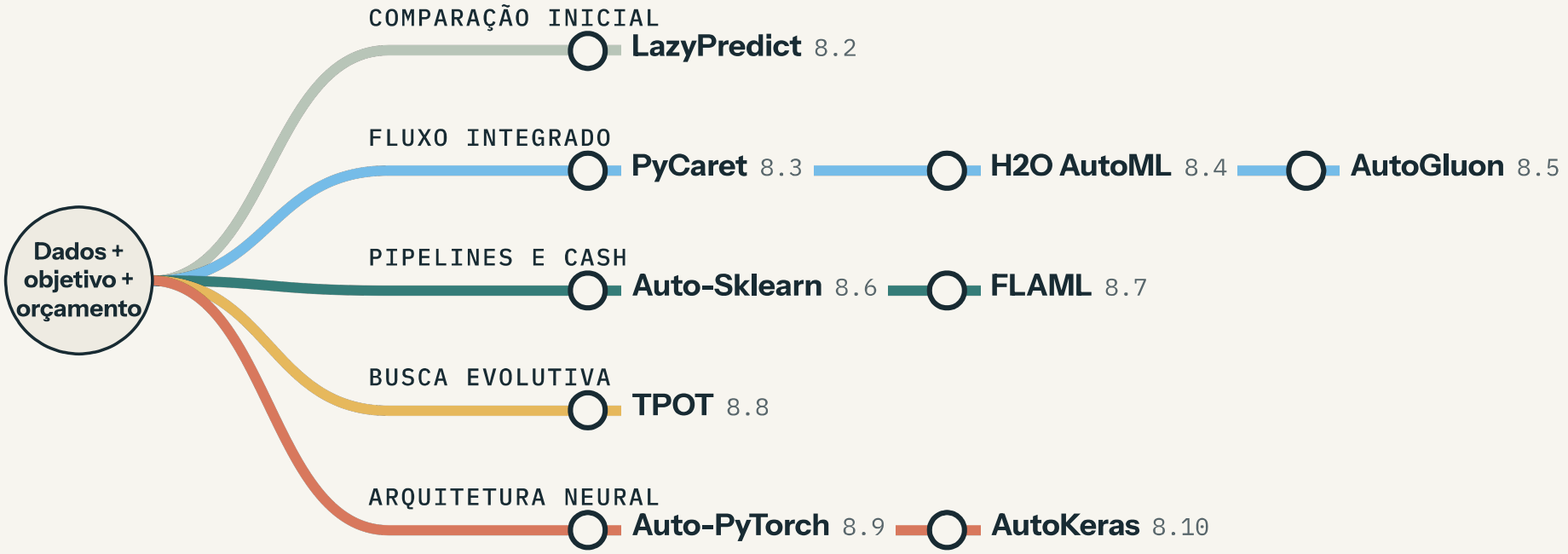


- **Exploração:** investigar configurações pouco conhecidas.
- **Aproveitamento:** investir nas regiões que já deram bons resultados.
- Regiões boas, desconhecidas no início da busca.

Com 5 min, o melhor encontrado é o losango verde. Com 1 h, a busca alcança a região da direita.

DADOS ILUSTRATIVOS

Nove ferramentas, cinco rotas pelo mesmo território



Linhas didáticas com cruzamentos reais. A ordem não é um ranking. Versões conferidas em 03/11/2026.

AULA 8.2

COMPARAÇÃO INICIAL

LazyPredict

VERSÃO CONSULTADA

0.3.0 no PyPI (15/03/2026)

VERIFICADO EM

03/11/2026

Treina muitos estimadores com configuração padrão e devolve uma tabela comparativa: uma linha de base em minutos.

O QUE AUTOMATIZA

- Treino e comparação de estimadores do scikit-learn e afins
- Pré-processamento padrão fixo
- Tabela de métricas e tempo de treino
- LazyClassifier, LazyRegressor; LazyForecaster desde a 0.3

LIMITAÇÕES

- Não otimiza hiperparâmetros
- Não constrói ensembles
- O número de modelos depende da versão e das bibliotecas instaladas

DOCUMENTAÇÃO OFICIAL

github.com/shankarpandala/lazypredict

INSTALAÇÃO E COMPATIBILIDADE

Extras opcionais para séries temporais e deep learning na 0.3. A tag v0.3.1 do GitHub não chegou ao PyPI; a 0.4.0, em preparação, remove o Python 3.9.

AULA 8.3

FLUXO INTEGRADO

PyCaret

VERSÃO CONSULTADA

3.3.2 estável · 4.0 em alfa

VERIFICADO EM

03/11/2026

Organiza o experimento em um fluxo low-code: preparar, comparar, ajustar, combinar, finalizar e salvar.

O QUE AUTOMATIZA

- setup: preparo e validação
- compare_models e tune_model
- blend_models (votação) e stack_models
- finalize_model e save_model

LIMITAÇÕES

- Forte dependência de versão e de dependências
- 3.x e 4.0 têm APIs diferentes: não misturar
- Interpretabilidade fora do escopo do alfa 4.0

INSTALAÇÃO E COMPATIBILIDADE

3.3.2: Python 3.9 a 3.11. 4.0 (alfa, pip install --pre): só orientada a objetos, ClassificationExperiment em pycaret.tasks, Python 3.11 a 3.13. Fork comunitário sktime/pycaret 3.5 para Python recente.

DOCUMENTAÇÃO OFICIAL

pycaret.org/docs

AULA 8.4

FLUXO INTEGRADO

H2O AutoML

VERSÃO CONSULTADA

h2o 3.46.0.12 (12/08/2026)

VERIFICADO EM

03/11/2026

Treina várias famílias em um cluster H2O, monta Stacked Ensembles e ordena tudo em um leaderboard.

O QUE AUTOMATIZA

- h2o.init() e dados em H2OFrame
- Modelos-base: GLM, DRF, GBM, XGBoost, Deep Learning
- Busca aleatória e Stacked Ensembles
- Leaderboard e exportação MOJO

LIMITAÇÕES

- Requer Java e um processo de cluster
- Memória do cluster limita os dados
- XGBoost do H2O não roda no Windows

DOCUMENTAÇÃO OFICIAL

docs.h2o.ai/h2o/latest-stable/h2o-docs/automl.html

INSTALAÇÃO E COMPATIBILIDADE

A partir da 3.46.0.12, quatro componentes saíram da distribuição open source para o H2O-3 Secure; confira o anúncio oficial antes da aula. A 3.46.0.11 segue disponível para quem precisa fixar versão.

AULA 8.5

FLUXO INTEGRADO

AutoGluon

VERSÃO CONSULTADA

1.6.3 estável (out/2026)

VERIFICADO EM

03/11/2026

Investe menos em busca e mais em ensembles: bagging, stacking em camadas e ensemble ponderado, controlados por presets.

O QUE AUTOMATIZA

- TabularPredictor(label).fit(presets, time_limit)
- Bagging com k folds e stacking em várias camadas
- WeightedEnsemble por seleção gulosa
- MultiModalPredictor e TimeSeriesPredictor

LIMITAÇÕES

- Presets altos consomem muita memória e disco
- Treino longo em presets de qualidade máxima
- Modelo final grande, com maior latência

DOCUMENTAÇÃO OFICIAL

auto.gluon.ai/stable/index.html

INSTALAÇÃO E COMPATIBILIDADE

Modelos fundacionais tabulares exigem extras, como autogluon.tabular[tabarena]. GPU opcional, recomendada para modelos multimodais.

AULA 8.6

PIPELINES E CASH

Auto-Sklearn

VERSÃO CONSULTADA

0.15.0 (última publicada)

VERIFICADO EM

03/11/2026

Resolve o CASH sobre pipelines do scikit-learn, começando com meta-learning e terminando com ensemble selection.

O QUE AUTOMATIZA

- Seleção de algoritmos e de pré-processadores
- Otimização bayesiana com SMAC
- Meta-learning para warm start
- Ensemble selection de Caruana

LIMITAÇÕES

- Funciona em Linux
- Não instala em Python 3.10+ sem contornos
- Sem lançamentos recentes: manutenção inativa

DOCUMENTAÇÃO OFICIAL

automl.github.io/auto-sklearn

INSTALAÇÃO E COMPATIBILIDADE

Use ambiente dedicado, como contêiner Linux com Python 3.9. O pacote de terceiros auto-sklearn2 não é o mesmo projeto: não tem meta-learning nem ensembles.

AULA 8.7

PIPELINES E CASH

FLAML

VERSÃO CONSULTADA

2.7.0 (18/09/2026)

VERIFICADO EM

03/11/2026

DOCUMENTAÇÃO OFICIAL

microsoft.github.io/FLAML

Escolhe estimadores e hiperparâmetros dentro de um time_budget, priorizando configurações baratas.

O QUE AUTOMATIZA

- AutoML().fit(X, y, task, metric, time_budget)
- Seleção de estimadores: LightGBM, XGBoost, RF e outros
- Busca consciente de custo: CFO e BlendSearch
- flaml.tune e configurações zero-shot (flaml.default)

LIMITAÇÕES

- Busca só entre os estimadores disponíveis
- Pouco pré-processamento automático
- Stacking apenas quando solicitado

INSTALAÇÃO E COMPATIBILIDADE

Python 3.10 a 3.13. Instale pip install "flaml[automl]" para o AutoML. O módulo autogen não faz mais parte do pacote.

AULA 8.8

BUSCA EVOLUTIVA

TPOT

VERSÃO CONSULTADA

1.1.0 (PyPI)

VERIFICADO EM

03/11/2026

DOCUMENTAÇÃO OFICIAL

epistasislab.github.io/tpot

Evolui uma população de pipelines por seleção, mutação e cruzamento, guiada pelo desempenho na validação.

O QUE AUTOMATIZA

- Programação genética sobre pipelines
- População, mutação, cruzamento e seleção
- Pipelines em grafo; seleção genética de atributos
- Otimização multiobjetivo; paralelismo com Dask

LIMITAÇÕES

- Busca cara em tempo e computação
- Resultados variam com a semente
- Pipelines finais podem ficar complexos

INSTALAÇÃO E COMPATIBILIDADE

TPOT 1.x incorporou o antigo TPOT2 e foi reescrito. A documentação sugere ambiente conda com Python 3.10. O export() que gerava um script .py é da série 0.12; na 1.x o resultado fica em fitted_pipeline_.

AULA 8.9

ARQUITETURA NEURAL

Auto-PyTorch

VERSÃO CONSULTADA

0.2.1 (última publicada)

VERIFICADO EM

03/11/2026

Otimiza conjuntamente arquitetura de rede e hiperparâmetros, com portfólio para warm start e ensembles.

O QUE AUTOMATIZA

- TabularClassificationTask().search()
- Arquitetura e hiperparâmetros com SMAC
- Portfólio guloso para warm start (meta-learning)
- Ensembles de redes; previsão de séries temporais

LIMITAÇÕES

- Custo alto de GPU e tempo
- Instalação voltada a Linux
- Sem lançamentos recentes: manutenção inativa

DOCUMENTAÇÃO OFICIAL

github.com/automl/Auto-PyTorch

INSTALAÇÃO E COMPATIBILIDADE

Documentação da 0.2.1: conda com Python 3.8, SWIG 3.0.x e scikit-learn anterior a 0.25. Use ambiente dedicado e fixe versões.

AULA 8.10

ARQUITETURA NEURAL

AutoKeras

VERSÃO CONSULTADA

3.0.0 (mais recente listada)

VERIFICADO EM

03/11/2026

Busca arquiteturas de redes para imagem, texto e dados estruturados com poucas linhas, sobre o Keras 3.

O QUE AUTOMATIZA

- ImageClassifier e TextClassifier
- StructuredDataClassifier (removido na 2.0, recolocado depois)
- Busca de arquitetura com KerasTuner e max_trials
- export_model() devolve um modelo Keras

LIMITAÇÕES

- Custo de GPU
- Compatibilidade de backend e de versão
- Espaço de busca restrito a redes neurais

INSTALAÇÃO E COMPATIBILIDADE

Keras 3 multi-backend: PyTorch, JAX ou TensorFlow; a documentação recomenda PyTorch (2.3 ou superior). Projeto reposicionado para experimentos rápidos em datasets pequenos.

DOCUMENTAÇÃO OFICIAL

autokeras.com

Foco, diferencial e limitação de cada ferramenta

FERRAMENTA	FOCO	DIFERENCIAL	LIMITAÇÃO	AULA
LazyPredict	Comparação inicial	Simplicidade, linha de base	Sem tuning nem ensembles	8.2
PyCaret	Fluxo low-code	Fluxo integrado	Dependência de versão	8.3
H2O AutoML	AutoML tabular	Leaderboard e Stacked Ensembles	Java e cluster	8.4
AutoGluon	Ensembles tabulares e multimodais	Presets, bagging e stacking	Custo e memória	8.5
Auto-Sklearn	CASH sobre pipelines	Meta-learning e SMAC	Compatibilidade e manutenção	8.6
FLAML	Busca econômica	Busca consciente de custo	Espaço de estimadores disponível	8.7
TPOT	Pipelines evolutivos	Programação genética	Custo da busca	8.8
Auto-PyTorch	Deep Learning tabular	NAS com portfólio	Recursos e manutenção	8.9
AutoKeras	Deep Learning	Busca de redes em Keras 3	Backend e versão	8.10

Sem ranking: as ferramentas têm objetivos diferentes.

Suporte por tipo de dado

FERRAMENTA	TABULAR	CATEGÓRICAS	IMAGENS	TEXTO	SÉRIES	REDES NEURAIS	PIPELINES
LazyPredict	✓	C	–	–	✓	L	L
PyCaret	✓	✓	–	–	✓	L	C
H2O AutoML	✓	✓	–	–	–	✓	–
AutoGluon	✓	✓	✓	✓	✓	✓	C
Auto-Sklearn	✓	✓	–	L	–	L	✓
FLAML	✓	✓	–	C	✓	C	L
TPOT	✓	C	–	–	–	L	✓
Auto-PyTorch	✓	✓	–	–	✓	✓	C
AutoKeras	C	C	✓	✓	–	✓	–

✓ Confirmado C Condicionado L Limitado – Não confirmado Documentação consultada em 03/11/2026.

Qual rota considerar em cada cenário

Comparar rapidamente vários modelos

LazyPredict · PyCaret

Fluxo low-code

PyCaret · AutoGluon

AutoML tabular com stacking

H2O AutoML · AutoGluon

Busca de pipelines

Auto-Sklearn · TPOT

Orçamento computacional pequeno

FLAML

Programação genética

TPOT

Busca de arquitetura neural

Auto-PyTorch · AutoKeras

Classificação de imagens

AutoKeras · AutoGluon

Classificação de texto

AutoKeras · AutoGluon

◇ Possibilidades, não respostas únicas. Ambiente, versão de Python, dados e restrições decidem.

Por que os Módulos 6 e 7 são pré-requisitos

SE O AUTOML ESCOLHER

Random Forest

você precisa entender bagging, max_depth e variância.

MÓDULO 7

SE O AUTOML ESCOLHER

Boosting

você precisa entender learning_rate, número de etapas, sobreajuste.

AULA 7.4

SE O AUTOML ESCOLHER

MLP

você precisa entender camadas, ativações, backpropagation.

MÓDULO 6

SE O AUTOML ESCOLHER

ROC AUC

você precisa entender ordenação de probabilidades, não acurácia.

MÉTRICAS

Regressão · Classificação · Métricas · Validação · Overfitting · Árvores · Random Forest · Boosting · Redes neurais · Hiperparâmetros

---

AutoML automatiza parte do processo, mas não substitui o conhecimento de quem constrói e avalia o modelo.

Limitações e riscos

Dados ruins
rótulos errados não se corrigem com busca

Dados insuficientes
poucas amostras, estimativas instáveis

Vazamento
colunas com informação do futuro ou do alvo

Validação inadequada
folds aleatórios em dados temporais

Métrica mal escolhida
otimiza o que pediu, não o que quis

Orçamento insuficiente
busca interrompida cedo demais

Custo computacional
ensembles caros para treinar e servir

Interpretabilidade
stacking difícil de explicar

Viés
padrões discriminatórios reproduzidos

Reprodutibilidade
sem semente e versão, resultados diferem

Implantação
dependências pesadas em produção

◇ **O AutoML pode otimizar a métrica e ainda produzir uma solução inadequada ao objetivo real.**

Quando evitar AutoML

1

Requisitos muito específicos

ex.: monotonicidade em relação à renda

2

Arquiteturas personalizadas

ex.: perda customizada de pesquisa

3

Sem validação confiável

a busca otimiza ruído

4

Dados inadequados

corrigir os dados vem antes

5

Orçamento insuficiente

um baseline bem pensado pode ser melhor

6

Ambiente incompatível

Windows sem Java, Python novo demais

7

Controle detalhado exigido

auditoria e regulação

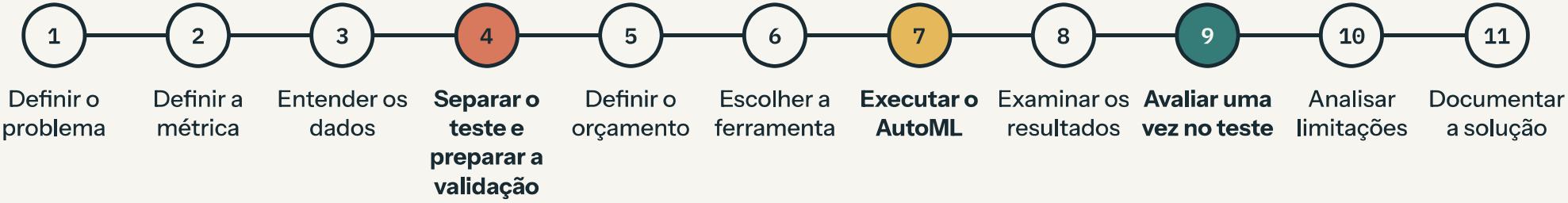
8

Solução simples já atende

regressão logística atinge o objetivo

Decisão de custo-benefício: ganho de desempenho contra custo de computação, manutenção e explicação.

Um fluxo responsável com AutoML



Estação 4: o teste é separado antes da busca e fica guardado.

Estação 9: o teste é usado uma única vez, depois de tudo escolhido.

Erros conceituais comuns · 1 a 8

	ERRO	INTERPRETAÇÃO CORRETA
01	✗ AutoML é uma única biblioteca	✓ É um campo com várias ferramentas e níveis de automação
02	✗ AutoML é apenas Grid Search	✓ Grid Search é uma das estratégias de busca
03	✗ AutoML e CASH são sinônimos	✓ CASH é um problema central dentro do AutoML
04	✗ Parâmetros e hiperparâmetros são iguais	✓ Parâmetros são aprendidos; hiperparâmetros são definidos antes
05	✗ Seleção de algoritmo é tuning	✓ Seleção escolhe a família; tuning ajusta a configuração
06	✗ Todas usam meta-learning	✓ Algumas: Auto-Sklearn, Auto-PyTorch; FLAML e AutoGluon com portfólios
07	✗ Todas fazem stacking	✓ H2O e AutoGluon por padrão; FLAML se solicitado; LazyPredict não
08	✗ Blending e stacking são iguais	✓ Blending usa holdout; stacking usa previsões out-of-fold

Erros conceituais comuns · 9 a 15

	ERRO	INTERPRETAÇÃO CORRETA
09	✗ Tuning é busca de arquitetura	✓ NAS escolhe a estrutura da rede
10	✗ AutoML dispensa validação	✓ A busca depende de uma validação confiável
11	✗ AutoML elimina vazamento	✓ Não detecta colunas que vazam o alvo
12	✗ Usar o teste durante a otimização	✓ O teste fica fora da busca e é usado uma vez
13	✗ Ignorar o orçamento	✓ O orçamento define o que pode ser explorado
14	✗ A mais automática é sempre melhor	✓ Mais automação traz mais custo e menos controle
15	✗ Versões não importam	✓ APIs mudam: PyCaret 4.0, AutoKeras 2.0, TPOT 1.0

◇ ROTA 09 · PRÁTICA OFICIAL · 20 MINUTOS

PRÁTICA

Atividade: mapa comparativo das nove ferramentas

1

Identificar

as nove ferramentas e o foco de cada uma

2

Classificar

em uma das cinco rotas

3

Mapear

o que cada uma automatiza

4

Vantagens

um diferencial por ferramenta

5

Limitações

uma limitação por ferramenta

6

Tipos de dados

matriz com C, L, ✓ ou –

7

Cenários

cinco estudos de caso

8

Infográfico

mapa em uma página

9

Justificar

escolhas com um mecanismo da aula

10

Revisar

trocar com outra dupla

0–8 MIN
etapas 1 a 6

8–14 MIN
cenários

14–18 MIN
infográfico

18–20 MIN
revisão em pares

Estudos de caso

A Preciso comparar rapidamente vários modelos.

B Quero um fluxo de ML com pouca programação manual.

C Quero buscar pipelines utilizando programação genética.

D Quero automatizar a arquitetura de redes neurais.

E Tenho orçamento de tempo limitado e preciso controlar o custo da busca.

Escolha uma ou duas ferramentas e justifique citando um mecanismo visto na aula.

Quiz: vinte perguntas

01 O que é AutoML?

02 O que significa CASH?

03 O que é seleção de algoritmos?

04 O que é otimização de hiperparâmetros?

05 O que é espaço de busca?

06 O que é meta-learning?

07 O que são meta-features?

08 O que é ensembling automático?

09 Diferença entre voting e stacking?

10 O que é weighted ensemble?

11 AutoML executa todas as etapas?

12 Qual a função do orçamento?

13 LazyPredict vs. AutoML abrangente?

14 O que é leaderboard?

15 O que é programação genética?

16 O que é Neural Architecture Search?

17 Por que o teste fica independente?

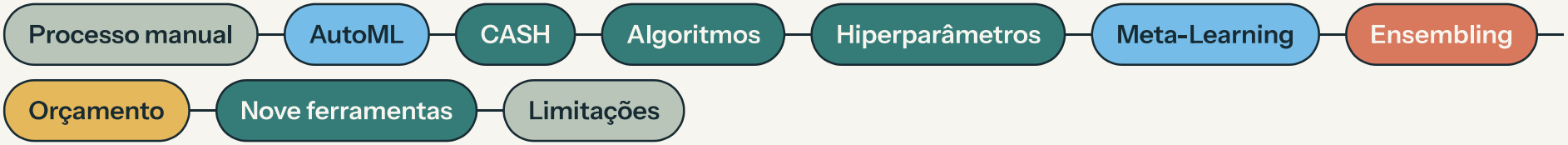
18 AutoML elimina vazamento?

19 Quando não utilizar AutoML?

20 Por que os Módulos 6 e 7 importam?

Respostas comentadas nas notas do professor e no gabarito.

O que mapeamos hoje e a próxima parada



Para cada ferramenta: que parte do problema ela automatiza, com que estratégia e com que custo?

PRÓXIMA
8.2

LazyPredict

Linha de base para o dataset da loja.